MySQL 8.0 Analytics Portfolio

Comprehensive E-Commerce Case Study & Complete Query Reference Standard

ENGINE	FOCUS AREAS	ADVANCED TECH	CORE FORMULA
MySQL 8.0+	**JOINS & CTEs**	**Window Functions**	**Qty × Unit Price**

Relational Database Schema Architecture

customers

`customer_id (PK)`, first_name, last_name
email, address, city, state_province

orders

`order_id (PK)`, `customer_id (FK)`
order_date

product_orders

`order_id (FK)`, `product_id (FK)`
quantity, unit_price

products

`product_id (PK)`, product_name
product_type

SECTION 1: CUSTOMER & PRODUCT AGGREGATIONS

Q1. Customer Revenue Analysis

INNER JOIN / GROUP BY

Find every customer who placed an order with order count, total items, and revenue sorted highest to lowest.

```sql
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS FullName,
       SUM(po.quantity) AS total_quantity_purchased,
       ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN product_orders po ON o.order_id = po.order_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_revenue DESC;
```

Q2. Product Performance

PRODUCT METRICS

Find the total revenue and total quantity sold for every ordered product.

```sql
SELECT p.product_id, p.product_name, p.product_type,
       SUM(po.quantity) AS total_quantity,
       ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
FROM products p
JOIN product_orders po ON p.product_id = po.product_id
GROUP BY p.product_id, p.product_name, p.product_type;
```

Q3. Product-Type Revenue Analysis

CATEGORY AGGREGATION

Find total revenue, distinct products sold, and total quantity across product types.

```sql
SELECT p.product_type,
       COUNT(DISTINCT po.product_id) AS number_of_distinct_products_sold,
       SUM(po.quantity) AS total_quantity_sold,
       ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
FROM products p
JOIN product_orders po ON p.product_id = po.product_id
GROUP BY p.product_type;
```

Q4. Customers With No Orders

SUBQUERY / NOT IN

Find customers who have never placed an order.

```sql
SELECT customer_id, first_name, last_name, city, state_province
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE customer_id IS NOT NULL
);
```

Q5. Products Never Ordered

SUBQUERY / UNORDERED

Find products that have never appeared in product_orders.

```sql
SELECT product_id, product_name, product_type
FROM products
WHERE product_id NOT IN (
    SELECT product_id FROM product_orders WHERE product_id IS NOT NULL
);
```

Q6. Order-Level Revenue

ORDER METRICS

For every order, calculate the total order value and total items.

```sql
SELECT o.order_id, o.order_date, c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       SUM(po.quantity) AS total_items,
       ROUND(SUM(po.quantity * po.unit_price), 2) AS order_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN product_orders po ON o.order_id = po.order_id
GROUP BY o.order_id, o.order_date, c.customer_id, c.first_name, c.last_name
ORDER BY c.customer_id;
```

SECTION 2: WINDOW FUNCTIONS & RANKING

Q7. Rank Customers by Revenue

DENSE_RANK()

Calculate each customer's total revenue and rank them using DENSE_RANK().

```sql
WITH total_rev AS (
    SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
           ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY c.customer_id, customer_name
)
SELECT *, DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS rnk
FROM total_rev;
```

Q8. Top 3 Products Within Each Product Type

ROW_NUMBER() / PARTITION

Calculate revenue for every product and return top 3 products within each product_type.

```sql
WITH cte1 AS (
    SELECT p.product_id, p.product_name, p.product_type,
           ROUND(SUM(po.quantity * po.unit_price)) AS revenue
    FROM product_orders po
    JOIN products p ON po.product_id = p.product_id
    GROUP BY p.product_id, p.product_name, p.product_type
),
row_num AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY product_type ORDER BY revenue DESC
    ) AS row_num
    FROM cte1
)
SELECT * FROM row_num WHERE row_num <= 3;
```

Q9. Running Monthly Revenue

RUNNING TOTAL

Calculate total revenue for each month and the cumulative (running) revenue over time.

```sql
WITH cte1 AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
           ROUND(SUM(po.quantity * unit_price), 2) AS monthly_revenue
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY month
)
SELECT month, monthly_revenue,
       SUM(monthly_revenue) OVER (ORDER BY month ASC) AS cumulative_revenue
FROM cte1;
```

Q10. Customer Order Sequence

ORDER SEQUENCE

For every order, show the customer's sequence number, order_id, order_date, and customer_id.

```sql
SELECT o.order_id, o.order_date, c.customer_id,
       ROW_NUMBER() OVER (
           PARTITION BY c.customer_id ORDER BY o.order_date ASC, o.order_id ASC
       ) AS order_number
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY c.customer_id;
```

Q11. Days Between Customer Orders

LAG() / DATEDIFF

Use LAG() to calculate the number of days between customer's current and previous order.

```
WITH cte1 AS (
    SELECT o.order_id, o.order_date, c.customer_id,
           LAG(o.order_date) OVER (
               PARTITION BY customer_id ORDER BY o.order_date ASC, o.order_id ASC
           ) AS prev_date
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
)
SELECT customer_id, DATEDIFF(order_date, prev_date) AS days_difference
FROM cte1
ORDER BY customer_id, order_date;
```

Q12. Order Contribution to Customer Revenue

PARTITION AGGREGATE

Calculate order value, customer total revenue, and percentage contribution per order.

```
WITH order_values AS (
    SELECT o.order_id, o.customer_id, o.order_date,
           SUM(po.quantity * po.unit_price) AS order_value
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.order_id, o.customer_id, o.order_date
)
SELECT order_id, customer_id, order_date,
       ROUND(order_value, 2) AS order_value,
       ROUND(SUM(order_value) OVER (PARTITION BY customer_id), 2) AS customer_total_revenue,
       ROUND(order_value * 100.0 / SUM(order_value) OVER (PARTITION BY customer_id), 2) AS
revenue_contribution_percentage
FROM order_values
ORDER BY customer_id, order_date, order_id;
```

Q13. Highest-Value Order for Every Customer

DENSE_RANK TOP-1

Return only the highest-value order for each customer (earlier order if tied).

```
WITH customerOrderValue AS (
    SELECT c.customer_id, o.order_id,
           ROUND(SUM(po.quantity * po.unit_price), 2) AS order_value
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.order_id, c.customer_id
),
ranked AS (
    SELECT *, DENSE_RANK() OVER (
        PARTITION BY customer_id ORDER BY order_value DESC, order_id ASC
    ) AS rnk
    FROM customerOrderValue
)
SELECT * FROM ranked WHERE rnk = 1;
```

SECTION 3: TIME SERIES & CUSTOMER SEGMENTATION

Q14. Monthly Revenue Growth

MoM GROWTH RATE

Calculate monthly revenue and compare each month with previous month using LAG().

```
WITH monthWise_revenue AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
           ROUND(SUM(po.quantity * po.unit_price), 2) AS monthly_revenue
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY month
),
preMonthRev AS (
    SELECT month, monthly_revenue,
           LAG(monthly_revenue) OVER (ORDER BY month) AS previous_month_revenue
    FROM monthWise_revenue
)
SELECT month, monthly_revenue, previous_month_revenue,
       ROUND(monthly_revenue - previous_month_revenue, 2) AS revenue_change,
       ROUND(((monthly_revenue - previous_month_revenue) / NULLIF(previous_month_revenue, 0)) * 100, 2)
AS growth_percentage
FROM preMonthRev;
```

Q15. Customer Purchase Segmentation

CASE WHEN SEGMENTATION

Segment customers into High Value (>=1000), Medium Value (>=500), and Low Value (<500).

```
WITH customer_revenue AS (
    SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
           ROUND(SUM(quantity * unit_price), 2) AS total_revenue,
           COUNT(DISTINCT o.order_id) AS order_count
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY c.customer_id, customer_name
)
SELECT *,
       (CASE WHEN total_revenue >= 1000 THEN 'High Value'
             WHEN total_revenue >= 500 AND total_revenue < 1000 THEN 'Medium Value'
             ELSE 'Low Value' END) AS segment
FROM customer_revenue;
```

Q16. Product Type Performance

MULTI-STAGE CTE

Calculate revenue for every product and summarize the results by product_type.

```
WITH product_revenue AS (
    SELECT p.product_id, p.product_type,
           SUM(po.quantity) AS total_quantity,
           SUM(po.quantity * po.unit_price) AS product_revenue
    FROM products p
    JOIN product_orders po ON p.product_id = po.product_id
    GROUP BY p.product_id, p.product_type
)
SELECT product_type, COUNT(product_id) AS products_sold, SUM(total_quantity) AS total_quantity,
       ROUND(SUM(product_revenue), 2) AS total_revenue, ROUND(AVG(product_revenue), 2) AS
average_product_revenue
FROM product_revenue
GROUP BY product_type ORDER BY total_revenue DESC;
```

Q17. Top 5 Customers in Each Month

MONTHLY TOP-N

Find top 5 customers by revenue in every month using ROW_NUMBER().

```sql
WITH customer_monthly_revenue AS (
    SELECT o.customer_id, DATE_FORMAT(o.order_date, '%Y-%m') AS order_month,
           SUM(po.quantity * po.unit_price) AS monthly_revenue
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.customer_id, DATE_FORMAT(o.order_date, '%Y-%m')
),
ranked_customers AS (
    SELECT customer_id, order_month, ROUND(monthly_revenue, 2) AS monthly_revenue,
           ROW_NUMBER() OVER (
               PARTITION BY order_month ORDER BY monthly_revenue DESC
           ) AS customer_rank
    FROM customer_monthly_revenue
)
SELECT customer_id, order_month, monthly_revenue, customer_rank
FROM ranked_customers
WHERE customer_rank <= 5
ORDER BY order_month, customer_rank;
```

Q18. Customers With Increasing Order Activity

FIRST_VALUE / LAST_VALUE

Find customers whose last order value is greater than their first order value.

```sql
WITH order_values AS (
    SELECT o.order_id, o.customer_id, o.order_date,
           SUM(po.quantity * po.unit_price) AS order_value
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.order_id, o.customer_id, o.order_date
),
customer_orders AS (
    SELECT customer_id,
           FIRST_VALUE(order_value) OVER (
               PARTITION BY customer_id ORDER BY order_date, order_id
           ) AS first_order_value,
           LAST_VALUE(order_value) OVER (
               PARTITION BY customer_id ORDER BY order_date, order_id
               ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
           ) AS last_order_value
    FROM order_values
)
SELECT DISTINCT customer_id, first_order_value, last_order_value
FROM customer_orders
WHERE last_order_value > first_order_value
ORDER BY customer_id;
```

Q19. Orders Above Monthly Average

MONTHLY BENCHMARK

Return orders whose value is greater than their month's average order value.

```sql
WITH order_values AS (
    SELECT o.order_id, o.order_date, DATE_FORMAT(o.order_date, '%Y-%m') AS order_month,
           ROUND(SUM(po.quantity * po.unit_price), 2) AS order_value
    FROM orders o
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.order_id, o.order_date
),
monthly_averages AS (
    SELECT order_month, ROUND(AVG(order_value), 2) AS monthly_avg_order_value
    FROM order_values
    GROUP BY order_month
)
SELECT ov.order_id, ov.order_date, ov.order_month, ov.order_value, ma.monthly_avg_order_value
FROM order_values ov
JOIN monthly_averages ma ON ov.order_month = ma.order_month
WHERE ov.order_value > ma.monthly_avg_order_value
ORDER BY ov.order_month, ov.order_id;
```

Q20. Customer Retention / Repeat Purchase Analysis

RETENTION ANALYSIS

Find customers with at least 2 orders and calculate days between first and last order.

```sql
WITH customer_orders AS (
    SELECT customer_id,
           MIN(order_date) AS first_order_date,
           MAX(order_date) AS last_order_date,
           COUNT(order_id) AS total_orders
    FROM orders
    GROUP BY customer_id
)
SELECT customer_id, first_order_date, last_order_date, total_orders,
       DATEDIFF(last_order_date, first_order_date) AS days_between_first_and_last_order
FROM customer_orders
WHERE total_orders >= 2
ORDER BY customer_id;
```

SECTION 4: ADVANCED SUBQUERIES & BENCHMARK METRICS

Q21. Customers Spending Above Average

SUBQUERY / AVG BENCHMARK

Find customers whose total revenue exceeds the average customer total revenue.

```sql
WITH customer_total_revenue AS (
    SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
           ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY c.customer_id, customer_name
)
SELECT * FROM customer_total_revenue
WHERE total_revenue > (SELECT AVG(total_revenue) FROM customer_total_revenue);
```

Q22. Products Above Average Revenue

SUBQUERY BENCHMARK

Find products whose total revenue is greater than the average revenue across all ordered products.

```sql
WITH product_revenue AS (
    SELECT p.product_id, p.product_name, ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
    FROM product_orders po
    JOIN products p ON po.product_id = p.product_id
    GROUP BY p.product_id, p.product_name
)
SELECT *, (SELECT ROUND(AVG(total_revenue), 2) FROM product_revenue) AS avg_product_revenue
FROM product_revenue
WHERE total_revenue > (SELECT ROUND(AVG(total_revenue), 2) FROM product_revenue);
```

Q23. Orders Above Customer Average Order Value

CORRELATED SUBQUERY

Find orders whose value is greater than the specific customer's average order value.

```sql
WITH order_values AS (
    SELECT o.order_id, c.customer_id, order_date, ROUND(SUM(quantity * unit_price), 2) AS order_value
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN product_orders po ON o.order_id = po.order_id
    GROUP BY o.order_id, c.customer_id, o.order_date
)
SELECT o1.order_id, o1.customer_id, o1.order_date, o1.order_value,
       (SELECT ROUND(AVG(o2.order_value), 2) FROM order_values o2 WHERE o2.customer_id = o1.customer_id)
AS customer_average_order_value
FROM order_values o1
WHERE o1.order_value > (SELECT AVG(o2.order_value) FROM order_values o2 WHERE o2.customer_id =
o1.customer_id);
```

Q24. Customers With At Least 3 Orders

SUBQUERY / HAVING

Find customers with at least 3 orders (solutions via subquery and GROUP BY HAVING).

```sql
-- Solution 1: Subquery approach
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       (SELECT COUNT(order_id) FROM orders o WHERE c.customer_id = o.customer_id) AS order_count
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(order_id) >= 3
)
ORDER BY customer_id;

-- Solution 2: Direct JOIN & HAVING
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       COUNT(order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, customer_name
HAVING order_count >= 3
ORDER BY c.customer_id;
```

Q25. Customers Who Bought a Specific Product Type

NESTED IN SUBQUERIES

Find customers who purchased at least one product from the 'oven' product type.

```sql
SELECT customer_id, CONCAT(first_name, ' ', last_name) AS customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders WHERE order_id IN (
        SELECT order_id FROM product_orders WHERE product_id IN (
            SELECT product_id FROM products WHERE product_type = 'oven'
        )
    )
);
```

Q26. Second-Highest Revenue Product

MAX SUBQUERY (NO LIMIT)

Find the product with the second-highest total revenue without using LIMIT/OFFSET.

```sql
SELECT product_id, product_name FROM (
    SELECT p.product_id, p.product_name, ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
    FROM products p JOIN product_orders po ON p.product_id = po.product_id
    GROUP BY p.product_id, p.product_name
) AS table0
WHERE total_revenue = (
    SELECT MAX(total_revenue) FROM (
        SELECT p.product_id, p.product_name, ROUND(SUM(po.quantity * po.unit_price), 2) AS total_revenue
        FROM products p JOIN product_orders po ON p.product_id = po.product_id
        GROUP BY p.product_id, p.product_name
    ) AS table_1
    WHERE total_revenue < (
        SELECT MAX(total_revenue) FROM (
            SELECT p.product_id, p.product_name, ROUND(SUM(po.quantity * po.unit_price), 2) AS
total_revenue
            FROM products p JOIN product_orders po ON p.product_id = po.product_id
            GROUP BY p.product_id, p.product_name
        ) AS table_2
    )
);
```